

Code Review Report

Language: python | Review mode: hybrid | Model: llama-3.3-70b-versatile | Generated: 2026-08-13 17:52 UTC

Overall Assessment

The code review has identified 6 findings with 2 High, 2 Medium, and 2 Low severity issues. The findings are related to complexity, security, maintainability, and performance.

Merge readiness: Not Ready

Code health: 52/100

Findings Summary

Total findings: 6

Risk score: 12

Severity breakdown: Low 2, Medium 2, High 2, Critical 0

Type	Severity	Line	Message	Recommendation
High Cyclomatic Complexity	High	1	Complexity = 36	Reduce nested decision points.
security	Medium	14	The code uses a hardcoded path for loading the corpus file. This could lead to issues if the file is not found or if the path is not correct.	Consider using a relative path or a configurable path for loading the corpus file.
maintainability	Low	23	The code uses a magic number (42) for seeding the random number generator. This could make the code less readable and maintainable.	Consider defining a named constant for the seed value to make the code more readable and maintainable.
performance	Low	105	The code uses a large table size (5,000,000) for negative sampling. This could lead to memory issues if the system has limited memory.	Consider using a smaller table size or a more efficient data structure for negative sampling.
security	High	154	The code uses a hardcoded learning rate (0.05) and a hardcoded number of epochs (60). This could lead to overfitting or underfitting if the hyperparameters are not tuned properly.	Consider using a hyperparameter tuning technique, such as grid search or random search, to find the optimal hyperparameters for the model.
maintainability	Medium	221	The code uses a complex and nested loop structure for training the model. This could make the code less readable and maintainable.	Consider breaking down the loop structure into smaller and more manageable functions to make the code more readable and maintainable.

Remediation Roadmap

Finding	Severity	Impact	Suggested Action
High Cyclomatic Complexity	High	Complexity = 36	Extract methods for each step of the Skip-gram model pipeline to reduce complexity
security	High	Hardcoded learning rate and number of epochs	Consider using a hyperparameter tuning technique to find the optimal hyperparameters for the model
security	Medium	Hardcoded path for loading the corpus file	Replace the hardcoded path with a relative path or a configurable path for loading the corpus file
maintainability	Medium	Complex and nested loop structure for training the model	Break down the loop structure into smaller and more manageable functions to make the code more readable and maintainable
maintainability	Low	Magic number (42) for seeding the random number generator	Define a named constant for the seed value to make the code more readable and maintainable
performance	Low	Large table size (5,000,000) for negative sampling	Consider using a smaller table size or a more efficient data structure for negative sampling

Source Code Snapshot

```
"""
skipgram_model.py
-----
A complete, from-scratch implementation of the Continuous Skip-gram model
with Negative Sampling (SGNS), as introduced by Mikolov et al. (2013) in
"Efficient Estimation of Word Representations in Vector Space" and
"Distributed Representations of Words and Phrases and their Compositionality".

No deep-learning framework (PyTorch / TensorFlow) is used. All forward and
backward computations are written explicitly with NumPy so every gradient
in this file matches, term for term, the derivation shown in Section III
of the accompanying IEEE paper.

Pipeline:
1. Read corpus.txt (built by build_corpus.py) and tokenize.
2. Build vocabulary + unigram frequency table.
3. Apply Mikolov subsampling of frequent words.
4. Generate (center, context) skip-gram training pairs with a dynamic window.
5. Build a negative-sampling noise distribution (unigram^0.75).
6. Train two embedding matrices (W_in, W_out) with mini-batch SGD.
7. Save embeddings, loss curve, nearest-neighbour tables, analogy results,
and a 2-D PCA projection for visualization.
"""

import json
import numpy as np
import matplotlib
```

```

matplotlib.use("Agg")
import matplotlib.pyplot as plt
from collections import Counter
from sklearn.decomposition import PCA

RNG = np.random.default_rng(42)

# -----
# 1. Load corpus & build vocabulary
# -----
with open("/home/claude/skipgram/outputs/corpus.txt") as f:
    sentences = [line.strip().split() for line in f if line.strip()]

word_counts = Counter(w for s in sentences for w in s)
vocab = sorted(word_counts, key=lambda w: -word_counts[w])
word2idx = {w: i for i, w in enumerate(vocab)}
idx2word = {i: w for w, i in word2idx.items()}
V = len(vocab)
total_tokens = sum(word_counts.values())

print(f"Vocabulary size (V): {V}")
print(f"Total tokens : {total_tokens}")

# -----
# 2. Mikolov subsampling of frequent words
#  $P(\text{discard word } w) = 1 - \sqrt{t / f(w)}$  where  $f(w)$  = relative frequency
# -----
t = 1e-3
freq = {w: c / total_tokens for w, c in word_counts.items()}
keep_prob = {w: min(1.0, (np.sqrt(freq[w] / t) + 1) * (t / freq[w])) for w in vocab}

subsampling_sentences = []
for s in sentences:
    new_s = [w for w in s if RNG.random() < keep_prob[w]]
    if len(new_s) > 1:
        subsampling_sentences.append(new_s)

# -----
# 3. Generate (center, context) training pairs with a dynamic window
# (dynamic window: an integer window size is sampled uniformly in
# [1, max_window] for every center word, exactly as in the original
# Word2Vec implementation, so nearby words are sampled more often)
# -----
MAX_WINDOW = 4
pairs = []
for s in subsampling_sentences:
    idxs = [word2idx[w] for w in s]
    n = len(idxs)
    for pos, center in enumerate(idxs):
        window = RNG.integers(1, MAX_WINDOW + 1)

```

```

start = max(0, pos - window)
end = min(n, pos + window + 1)
for ctx_pos in range(start, end):
    if ctx_pos != pos:
        pairs.append((center, idxs[ctx_pos]))

pairs = np.array(pairs, dtype=np.int64)
RNG.shuffle(pairs)
print(f"Total training pairs: {len(pairs)}")

# -----
# 4. Negative sampling noise distribution: unigram frequency raised to 0.75
# -----
unigram_counts = np.array([word_counts[idx2word[i]] for i in range(V)],
dtype=np.float64)
noise_dist = unigram_counts ** 0.75
noise_dist /= noise_dist.sum()

# Precompute a large sampling table for O(1) negative sampling
TABLE_SIZE =
...
```